

Employee Management System

ABSTRACT

Microservices-Based Application

The Employee Management System is a microservices-based application designed to centralize and manage employee-related organizational operations. The system provides separate services for employee management, department management, salary and payroll, leave management, attendance, support tickets, and notifications. The architecture is intended to provide clear service boundaries, independent data ownership, secure access control, and maintainable business functionality.

System Overview

The system supports different organizational roles including Administrator, HR, Manager, Employee, and Support Staff. Role-based authorization is used to control access to operations and employee-specific information. Employees can manage or view their permitted profile, salary, payslip, leave, attendance, overtime, support tickets, and notifications, while authorized administrative users manage organizational and operational activities.

Core Services

Service	Responsibility
Employee Service	Employee records, CRUD operations, profile-related operations and employee search.
Department Service	Department creation, viewing, updating, deletion and assignment constraints.
Salary Service	Salary, components, revisions, bonuses, overtime, payroll and payslips.
Leave Service	Leave application, history, approval, rejection and status tracking.
Attendance Service	Employee check-in, check-out, attendance records and history.
Support Service	Support ticket creation, assignment, status updates, resolution and closure.
Notification Service	Event-based notification creation, viewing and read-status management.

Architecture and Technology

- Microservices architecture with independently maintainable business services.
- API Gateway as the client-facing entry point for routing.
- Eureka for service discovery.
- PostgreSQL with a database-per-service approach.
- Entity Framework Core for data access.

- REST APIs for synchronous communication.
- Apache Kafka for asynchronous event-driven communication.
- Security using authentication, JWT, RBAC and resource-level authorization.
- Swagger/OpenAPI for API documentation and Bruno for API testing.
- xUnit for unit testing and support for integration and authorization testing.

Key Objectives

- Centralize employee and organizational information.
- Provide controlled access according to organizational roles.
- Separate business capabilities into independently maintainable services.
- Protect employee, salary and payroll information.
- Support operational workflows for leave, attendance, overtime, payroll and support.
- Provide documented, versioned and testable APIs.
- Use event-driven communication for supported notification workflows.

Conclusion

The Employee Management System provides a structured platform for managing core employee and organizational processes through a microservices architecture. By separating business capabilities into dedicated services and enforcing service-level data ownership, security, communication, and testing requirements, the system establishes a modular foundation for implementation and future maintenance. The scope is limited to the defined Employee, Department, Salary, Leave, Attendance, Support, and Notification services.